Chapter 19. Securing Applications in Kubernetes

Providing a secure platform to run your workloads is critical for Kubernetes to be broadly used in production. Thankfully, Kubernetes ships with many different security-focused APIs that allow you to construct a secure operating environment. The challenge is that there are many different security APIs, and you have to declaratively opt-in to use them. Using these security-focused APIs can be cumbersome and convoluted, which makes it difficult to achieve your desired security goals.

It's important to understand the following two concepts when securing Pods in Kubernetes: defense in depth and principle of least privilege. *Defense in depth* is a concept where you use multiple layers of security controls across your computing systems that include Kubernetes. The *principle of least privilege* means giving your workloads access only to resources that are required for them to operate. Both these concepts are not destinations, but constantly applied to the ever-changing computing system landscape.

In this chapter, we will take a look at security-focused Kubernetes APIs that can be incrementally applied to help secure your workloads at the Pod level.

Understanding SecurityContext

At the core of securing Pods is SecurityContext, which is an aggregation of all security-focused fields that may be applied at both the Pod and container specification level. Here are some example security controls covered by SecurityContext:

- User permissions and access control (e.g., setting User ID and Group ID)
- Read-only root filesystem
- Allow privilege escalation
- Seccomp, AppArmor, and SELinux profile and label assignments
- Run as privileged or unprivileged

Let's take a look at an example Pod with a SecurityContext defined in Example 19-1.

Example 19-1. kuard-pod-securitycontext.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        privileged: false
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

You can see in this example that there is a SecurityContext at both the Pod and the container level. Many of the security controls can be applied at both of these levels. In the case that they are applied in both, the container level configuration takes precedence. Let's take a look at fields we have defined in the Pod specification in this example and the impact they have on securing your workload:

runAsNonRoot

The Pod or container must run as a nonroot user. The container will fail to start if it is running as a root user. Running as a nonroot user is considered best practice as many misconfigurations and exploits happen via the container runtime conflating the container process running as the root user with the host root user. This can be set at both the PodSecurityContext and the SecurityContext. The kuard container image is configured to run as user "nobody" as defined in the Dockerfile. It's always best practice to run your container as a nonroot user; however, if you are running a container downloaded from another source that doesn't explicitly set the container user, you may have to extend the original Dockerfile to do so. This method doesn't always work, as the application may have other requirements that needs to be considered.

runAsUser/runAsGroup

This setting overrides the user and group that the container process is run as. Container images may have this configured as part of the Dockerfile.

fsgroup

Configures Kubernetes to change the group of all files in a volume when they are mounted into a Pod. An additional field, `fsGroupChangePolicy`, may be used to configure the exact behavior.

allowPrivilegeEscalation

Configures whether a process in a container can gain more privileges than its parent. This is a common vector for attack, and it's important to explicitly set this to false. It's also important to understand that this will be set to true if `privileged: true` is set.

privileged

Runs the container as privileged, which elevates the container to the same permissions as the host.

readOnlyRootFilesystem

Mounts the container root filesystem to read-only. This is a common attack vector and is best practice to enable. Any data or logs that the workloads need write access to can be mounted via a volume.

The fields in this example aren't a complete list of all the security controls available; however, they represent a good starting point when working with SecurityContext. We will cover some more in context later in this chapter.

Let's now create the Pod by saving this example to a file called *kuard-pod-securitycontext.yaml*. We will demonstrate how the SecurityContext configuration is being applied to a running Pod. Create the Pod using the following command:

```
$ kubectl create -f kuard-pod-securitycontext.yaml
pod/kuard created
```

Now we'll start a shell inside the kuard container and check which user ID and group ID the processes are running as:

```
$ kubectl exec -it kuard -- ash
/ $ id
uid=1000 gid=3000 groups=2000
/ $ ps
PID   USER     TIME  COMMAND
    1 1000      0:00 /kuard
   30 1000      0:00 ash
   37 1000      0:00 ps
/ $ touch file
touch: file: Read-only file system
```

We can see that the shell that we started, `ash`, is running as user ID (uid) 1000, group ID (gid) 3000, and is in group 2000. We can also see that the `kuard` process is running as user 1000 as defined by the SecurityContext in the Pod specification. We also confirmed that we aren't able to create any new files because the container is read-only. If you only apply the following changes to you workloads, you're already off to a great start.

We will now introduce several other security controls covered by SecurityContext, which enable even more fine-grained control over what access and privileges your workloads have. First, we will introduce the operating system level security controls and then how to configure them via SecurityContext. It's important to note that many of these controls are host operating system dependent. This means that they may only apply to containers running on Linux operating systems as opposed to other supported Kubernetes operating systems like Windows. Here are a list of the core set of operating system controls that are covered by SecurityContext:

Capabilities

Allow either the addition or removal of groups of privilege that may be required for a workload to operate. For example, your workload may configure the host's network configuration. Rather than configuring the Pod to be privileged, which is effectively host root access, you could add the specific capability to configure the host networking configuration (NET_ADMIN is the specific capability name). This follows the principal of least privilege.

AppArmor

Controls which files processes can access. AppArmor profiles can be applied to containers via the addition of an annotation of `container.apparmor.security.beta.kubernetes.io/<container_name>: <profile_ref>` to the Pod specification. Acceptable values for `<profile ref>` include

`runtime/default`, `localhost/<path to profile>`, and `unconfined`. The default is `unconfined`, which explicitly sets no profile to be applied.

Seccomp

Seccomp (secure computing) profiles allow the creation of syscall filters. These filters allow specific syscalls to be allowed or blocked, which limits the surface area of the Linux kernel that is exposed to the processes in the Pods.

SELinux

Defines access controls for files and processes. SELinux operators use labels that are grouped together to create a security context (not to be mistaken with a Kubernetes SecurityContext), which is used to limit access to a process. By default, Kubernetes allocates a random SELinux context for each container; however, you may choose to set one via SecurityContext.

NOTE

Both AppArmor and seccomp have the ability to set the runtime default profile to be used. Each container runtime ships with default AppArmor and seccomp profiles that have been carefully curated to reduce the attack surface area by removing syscalls and file access that are known to be attack vectors or aren't commonly used by applications. These defaults are rarely workload impacting and offer a great starting point.

To demonstrate how these security controls are applied to a Pod, we will use a tool called amicontained ("Am I contained") written by Jess Frazelle. Save the Pod specification in Example 19-2 to a file called *amicontained-pod.yaml*. The first Pod has no SecurityContext applied and will be used to show which security controls are applied to a Pod by default. Note that your output may look different because different Kubernetes distributions and managed services provide different defaults.

Example 19-2. amicontained-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: amicontained
spec:
  containers:
    - image: r.j3ss.co/amicontained:v0.4.9
      name: amicontained
```

```
    command: [ "/bin/sh", "-c", "--" ]
    args: [ "amicontained" ]
```

Create the `amicontainer` Pod:

```
$ kubectl apply -f amicontained-pod.yaml
pod/amicontained created
```

Let's review the Pod logs to examine the output of the `amicontained` tool:

```
$ kubectl logs amicontained
Container Runtime: kube
Has Namespaces:
      pid: true
      user: false
AppArmor Profile: docker-default (enforce)
Capabilities:
      BOUNDING -> chown dac_override fowner fsetid kill setgid setuid
      setpcap net_bind_service net_raw sys_chroot mknod audit_write
      setfcap
Seccomp: disabled
Blocked Syscalls (21):
      SYSLOG SETPGID SETSID VHANGUP PIVOT_ROOT ACCT SETTIMEOFDAY UMOUNT2
      SWAPON SWAPOFF REBOOT SETHOSTNAME SETDOMAINNAME INIT_MODULE
      DELETE_MODULE LOOKUP_DCOOKIE KEXEC_LOAD FANOTIFY_INIT
      OPEN_BY_HANDLE_AT FINIT_MODULE KEXEC_FILE_LOAD
Looking for Docker.sock
```

From the output above we see that the AppArmor runtime default is being applied. We also see the capabilities that are allowed by default along with seccomp being disabled. Finally, we see that a total of 21 syscalls are being blocked by default. Now that we have a baseline, let's apply seccomp, AppArmor, and Capabilities security controls to the Pod specification. Create a file called *amicontained-pod-securitycontext.yaml* with the contents of Example 19-3.

Example 19-3. amicontained-pod-securitycontext.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: amicontained
```

```
  annotations:
    container.apparmor.security.beta.kubernetes.io/amicontained: "runtime/default"
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
    seccompProfile:
      type: RuntimeDefault
  containers:
    - image: r.j3ss.co/amicontained:v0.4.9
      name: amicontained
      command: [ "/bin/sh", "-c", "--" ]
      args: [ "amicontained" ]
      securityContext:
        capabilities:
          add: ["SYS_TIME"]
          drop: ["NET_BIND_SERVICE"]
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        privileged: false
```

First, we need to delete the existing `amicontained` Pod:

```
$ kubectl delete pod amicontained
pod "amicontained" deleted
```

Now we can create the new Pod with the SecurityContext applied. We are specifically declaring that the runtime default AppArmor and seccomp profiles be applied. In addition, we have added and dropped a Capability:

```
$ kubectl apply -f amicontained-pod-securitycontext.yaml
pod/amicontained created
```

Let's again review the Pod logs to examine the output of the `amicontained` tool:

```
$ kubectl logs amicontained
Container Runtime: kube
Has Namespaces:
        pid: true
```

```
        user: false
    AppArmor Profile: docker-default (enforce)
    Capabilities:
        BOUNDING -> chown dac_override fowner fsetid kill setgid setuid setpcap
        net_raw sys_chroot sys_time mknod audit_write setfcap
    Seccomp: filtering
    Blocked Syscalls (67):
        SYSLOG SETUID SETGID SETPGID SETSID SETREUID SETREGID SETGROUPS
        SETRESUID SETRESGID USELIB USTAT SYSFS VHANGUP PIVOT_ROOT_SYSCTL ACCT
        SETTIMEOFDAY MOUNT UMOUNT2 SWAPON SWAPOFF REBOOT SETHOSTNAME
        SETDOMAINNAME IOPL IOPERM CREATE_MODULE INIT_MODULE DELETE_MODULE
        GET_KERNEL_SYMS QUERY_MODULE QUOTACTL NFSSERVCTL GETPMSG PUTPMSG
        AFS_SYSCALL TUXCALL SECURITY LOOKUP_DCOOKIE VSERVER MBIND SET_MEMPOLICY
        GET_MEMPOLICY KEXEC_LOAD ADD_KEY REQUEST_KEY KEYCTL MIGRATE_PAGES
        FUTIMESAT UNSHARE MOVE_PAGES PERF_EVENT_OPEN FANOTIFY_INIT
        NAME_TO_HANDLE_AT OPEN_BY_HANDLE_AT SETNS PROCESS_VM_READV
        PROCESS_VM_WRITEV KCMP FINIT_MODULE KEXEC_FILE_LOAD BPF USERFAULTFD
        PKEY_MPROTECT PKEY_ALLOC PKEY_FREE
    Looking for Docker.sock
```

SecurityContext Challenges

As you can see, there is a lot to understand to use a SecurityContext, and it is not easy to apply a baseline set of security controls by directly configuring all fields of every Pod. The creation and management of AppArmor, seccomp, and SELinux profiles and contexts is not easy and is error prone. The cost of an error is breaking the ability for an application to perform its function. There are several tools out there that create a way to generate a seccomp profile from a running Pod, which can then be applied using SecurityContext. One such project is the Security Profiles Operator, which makes it easy to generate and manage Seccomp profiles. We will now take a look at other security APIs that make the management of how SecurityContext is applied consistent across a cluster.

Pod Security

Now that we've taken a look at SecurityContext as a way to manage security controls applied to Pods and containers, we will cover how to make sure that a set of Security Context values are applied at scale. Kubernetes has a now-deprecated PodSecurityPolicy (PSP) API, which enabled both validation and mutation. *Validation* will not allow the creation of Kubernetes resources unless they have a specific SecurityContext applied. *Mutation*, on the other hand, will change Kubernetes resources and apply a specific

SecurityContext based on criteria applied via the PSP. Given that PSP is deprecated and will be removed in Kubernetes v1.25, we will not cover it in depth but will instead cover its successor, Pod Security. One of the main differences between Pod Security and its predecessor is that Pod Security only performs validation and not mutation. If you want to learn more about mutation, we encourage you to take a look at Chapter 20.

What Is Pod Security?

Pod Security allows you to declare different security profiles for Pods. These security profiles are known as Pod Security Standards and are applied at the namespace level. Pod Security Standards are a collection of security-sensitive fields in a Pod specification (including, but not limited to, SecurityContext) and their associated values. There are three different standards that range from restricted to permissive. The idea is that you can apply a general security posture to all Pods in a given namespace. The three Pod Security Standards are as follows:

Baseline

Most common privilege escalation while enabling easier onboarding.

Restricted

Highly restricted, covering security best practices. May cause workloads to break.

Privileged

Open and unrestricted.

WARNING

Pod Security is currently a beta feature as of Kubernetes v1.23 and may be subject to change.

Each Pod Security Standard defines a list of fields in the Pod specification and their allowed values. Here are some fields that are covered by these standards:

- `spec.securityContext`
- `spec.containers[*].securityContext`
- `spec.containers[*].ports`
- `spec.volumes[*].hostPath`

You can view the complete list of fields covered by each of the Pod Security Standards in the offical documentation.

Each standard is applied to a namespace using a given mode. There are three modes a policy may be applied to. They are as follows:

Enforce

Any Pods that violate the policy will be denied.

Warn

Any Pods that violate the policy will be allowed, and a warning message will be displayed to the user.

Audit

Any Pods that violate the policy will generate an audit message in the audit log.

Applying Pod Security Standards

Pod Security Standards are applied to a namespace using labels as follows:

- Required: `pod-security.kubernetes.io/<MODE>: <LEVEL>`
- Optional: `pod-security.kubernetes.io/<MODE>-version: <VERSION>` (defaults to latest)

The namespace in Example 19-4 illustrates how you may use multiple modes to enforce at one standard (baseline in this example) and audit and warn at another (restricted). Using multiple modes allows you to deploy a policy with a lower security posture and audit which workloads violate a standard with a more restricted policy. You can then remediate the policy violations before enforcing the more restricted standard. You can also pin a mode to a specific version, e.g., v1.22. This allows the policy standards to change with each Kubernetes release and allows you to pin a specific version. In Example 19-4, we are enforcing the baseline standard and both warning and auditing the restricted standard. All modes are pinned to v1.22 of the standard.

Example 19-4. baseline-ns.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: baseline-ns
  labels:
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/enforce-version: v1.22
```

```
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/audit-version: v1.22
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/warn-version: v1.22
```

Deploying a policy for the first time can be a daunting task. Thankfully, Pod Security has made it easy to see which existing workloads violate a Pod Security Standard with a single dry-run command:

```
$ kubectl label --dry-run=server --overwrite ns \
  --all pod-security.kubernetes.io/enforce=baseline
Warning: kuard: privileged
namespace/default labeled
namespace/kube-node-lease labeled
namespace/kube-public labeled
Warning: kube-proxy-vxjwb: host namespaces, hostPath volumes, privileged
Warning: kube-proxy-zxqzz: host namespaces, hostPath volumes, privileged
Warning: kube-apiserver-kind-control-plane: host namespaces, hostPath volumes
Warning: etcd-kind-control-plane: host namespaces, hostPath volumes
Warning: kube-controller-manager-kind-control-plane: host namespaces, ...
Warning: kube-scheduler-kind-control-plane: host namespaces, hostPath volumes
namespace/kube-system labeled
namespace/local-path-storage labeled
```

This command evaluates all Pods on a Kubernetes cluster against the baseline Pod Security Standard and reports violations as warning messages in the output.

Let's see Pod Security in action. Create a file called *baseline-ns.yaml* with the content in Example 19-5.

Example 19-5. baseline-ns.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: baseline-ns
  labels:
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/enforce-version: v1.22
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/audit-version: v1.22
```

```
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/warn-version: v1.22
```

```
$ kubectl apply -f baseline-ns.yaml
namespace/baseline-ns created
```

Create a file called *kuard-pod.yaml* with the content in Example 19-6.

Example 19-6. kuard-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
  labels:
    app: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

Create the Pod and review the output with the following command:

```
$ kubectl apply -f kuard-pod.yaml --namespace baseline-ns
Warning: would violate "v1.22" version of "restricted" PodSecurity profile:
allowPrivilegeEscalation != false (container "kuard" must set
securityContext.allowPrivilegeEscalation=false), unrestricted capabilities
(container "kuard" must set securityContext.capabilities.drop=["ALL"]),
runAsNonRoot != true (pod or container "kuard" must set securityContext.
runAsNonRoot=true), seccompProfile (pod or container "kuard" must set
securityContext.seccompProfile.type to "RuntimeDefault" or "Localhost")
pod/kuard created
```

In this output, you can see that the Pod was successfully created; however, it violated the restricted Pod Security Standard, and the details of the violations are provided in the

output so that you can remediate. We can also see the message in the API server audit log because we configured the audit mode:

```
{"kind":"Event","apiVersion":"audit.k8s.io/v1","level":"Metadata","auditID":"...
```

Pod Security is a great way to manage the security posture of your workloads by applying policy at the namespace level and allowing Pods to be created only if they don't violate the policy. It's flexible and offers different prebuilt policies from permissive to restricted along with tooling to easily roll out policy changes without the risk of breaking workloads.

Service Account Management

Service accounts are Kubernetes resources that provide an identity to workloads that run inside Pods. RBAC can be applied to service accounts to control what resources, via the Kubernetes API, the identity has access to. Please see Chapter 14 to learn more. If your application doesn't require access to the Kubernetes API, you should disable access following the least privilege principal. By default, Kubernetes creates a default service account in each namespace, which is automatically set as the service account for all Pods. This service account contains a token that is automounted in each Pod and is used to access the Kubernetes API. To disable this behavior, you must add `automountServiceAccountToken: false` to the service account configuration. Example 19-7 demonstrates how this can be done for the default service account. This must be done in each namespace.

Example 19-7. service-account.yaml

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: default
automountServiceAccountToken: false
```

Service accounts are often overlooked when considering Pod security; however, they allow direct access to the Kubernetes API and, without adequate RBAC, could allow an attacker access to Kubernetes. It's important to understand how to limit access by making a simple change to how service account tokens are handled.

Role-Based Access Control

We would be remiss not to mention Kubernetes role-based access control (RBAC) in a chapter about securing Pods. Everything you need to know about RBAC can be found in Chapter 14 and can be applied to complement you workload's security posture.

RuntimeClass

Kubernetes interacts with the container runtime on the node's operating system via the Container Runtime Interface (CRI). The creation and standardization of this interface has allowed for an ecosystem of container runtimes to exist. These container runtimes may offer different levels of isolation, which include stronger security guarantees based on how they are implemented. Projects like Kata Containers, Firecracker, and gVisor are based on different isolation mechanisms from nested virtualization to more sophisticated syscall filtering. These security and isolation guarantees provide a Kubernetes administrator the flexibility to allow users to select a container runtime based on their workload type. For example, if your workload needs stronger security guarantees, then you can choose to run in a Pod that uses a different container runtime.

The RuntimeClass API was introduced to allow container runtime selection. It allows users to select one of a supported list of container runtimes in the cluster. Figure 19-1 depicts how RuntimeClass functions.

NOTE

Different RuntimeClasses must be configured by a cluster administrator and may required specific `nodeSelectors` or `tolerations` on your workload to be scheduled to the correct node.

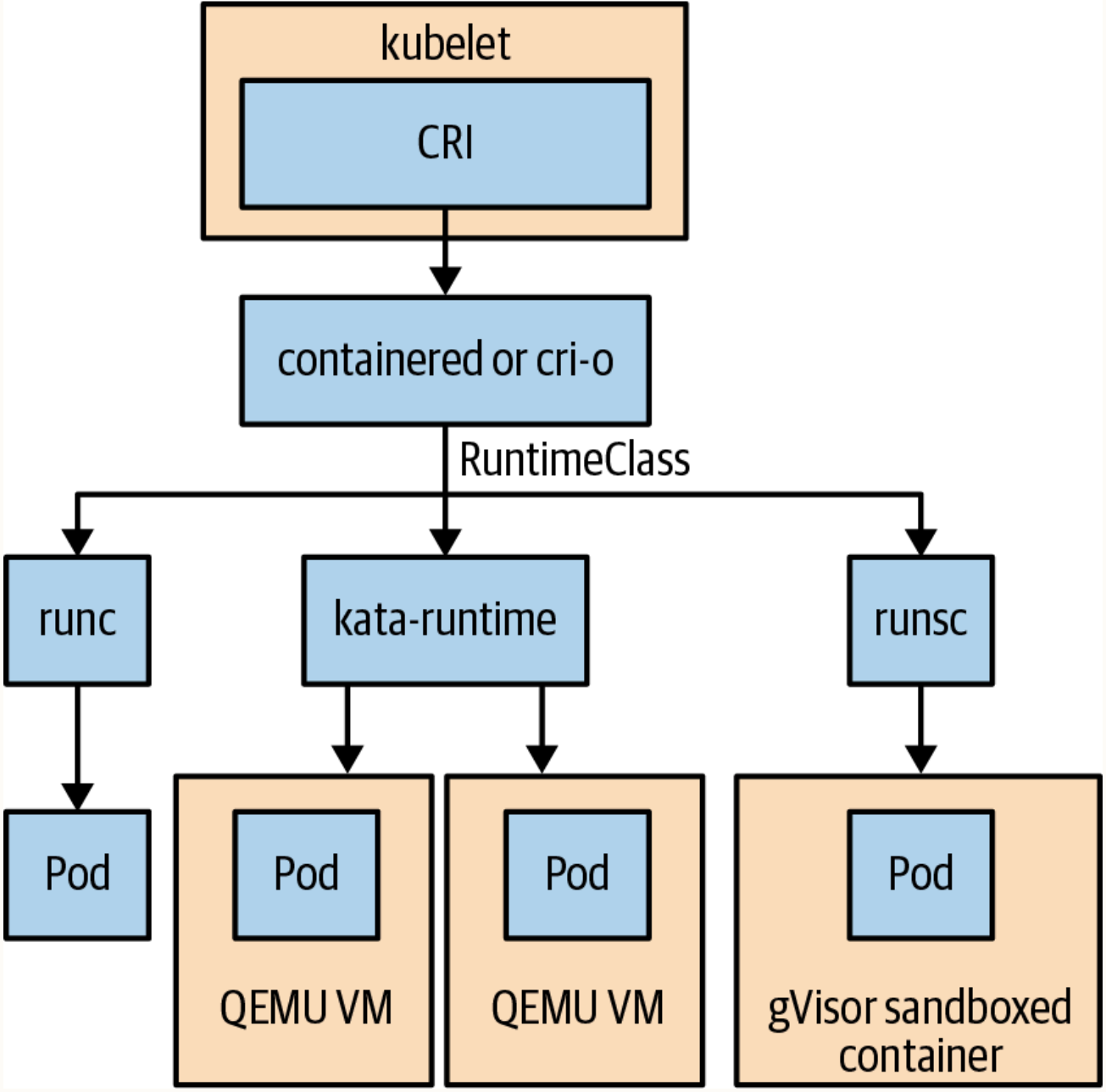


Figure 19-1. `RuntimeClass` flow diagram

You can use a RuntimeClass by specifying `runtimeClassName` in the Pod specification. Example 19-8 is an example Pod that specifies a RuntimeClass.

Example 19-8. kuard-pod-runtimeclass.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
  labels:
    app: kuard
spec:
  runtimeClassName: firecracker
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
```

```
    ports:
     - containerPort: 8080
       name: http
       protocol: TCP
```

RuntimeClass allows users to select different container runtimes that may have different security isolation. Using RuntimeClass can help complement the overall security of your workloads, especially if workloads are processing sensitive information or running untrusted code.

Network Policy

Kubernetes also has a Network Policy API that allows you to create both ingress and egress network policies for your workload. Network policies are configured using labels that allow you to select specific Pods and define how they can communicate with other Pods and endpoints. A Network Policy like Ingress doesn't actually ship with an associated Kubernetes controller. This means that you can create Network Policy resources but if you haven't installed a controller that acts upon the creation of Network Policy resources, then they will not be enforced. Network Policy resources are implemented by network plug-ins, such as Calico, Cilium, and Weave Net.

The Network Policy resource is namespaced and is structured with the `podSelector`, `policyTypes`, `ingress`, and `egress` sections with the only required field being `podSelector`. If the `podSelector` field is empty, the policy matches all Pods in a namespace. This field may also contain a `matchLabels` section, which functions in the same way as a Service resource, allowing you to add a set of labels to match a specific set of Pods.

There are several idiosyncrasies when using Network Policy that you need to be aware of. If a Pod is matched by any Network Policy resource, then any ingress or egress communication must be explicitly defined, otherwise it will be blocked. If a Pod matches multiple Network Policy resources, then the policies are additive. If a Pod isn't matched by any Network Policy, then traffic is allowed. This decision was intentionally made to ease onboarding of new workloads. If you do, however, want all traffic to be blocked by default, you can create a default deny rule per namespace. Example 19-9 shows a default deny rule that can be applied per namespace.

Example 19-9. networkpolicy-default-deny.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Let's walk through an example set of network policies to demonstrate how you can use them to secure your workloads. First, create a namespace to test using the following command:

```
$ kubectl create ns kuard-networkpolicy
namespace/kuard-networkpolicy created
```

Create a file named *kuard-pod.yaml* with the contents of Example 19-10.

Example 19-10. kuard-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: kuard
  labels:
    app: kuard
spec:
  containers:
    - image: gcr.io/kuar-demo/kuard-amd64:blue
      name: kuard
      ports:
        - containerPort: 8080
          name: http
          protocol: TCP
```

Create the `kuard` Pod in the `kuard-networkpolicy` namespace:

```
$ kubectl apply -f kuard-pod.yaml \
  --namespace kuard-networkpolicy
```

```
pod/kuard created
```

Expose the `kuard` Pod as a service:

```
$ kubectl expose pod kuard --port=80 --target-port=8080 \
 --namespace kuard-networkpolicy
pod/kuard created
```

Now we can use `kubectl run` to spin up a Pod to test as our source and test access to the `kuard` Pod without applying any Network Policy:

```
$ kubectl run test-source --rm -ti --image busybox /bin/sh \
 --namespace kuard-networkpolicy
If you don't see a command prompt, try pressing enter.
/ # wget -q kuard -O -
<!doctype html>

<html lang="en">
<head>
 <meta charset="utf-8">

 <title><KUAR Demo></title>
...
```

We can successfully connect to the `kuard` Pod from our test-source Pod. Now let's apply a default deny policy and test again. Create a file called *networkpolicy-default-deny.yaml* with the contents of Example 19-11.

Example 19-11. networkpolicy-default-deny.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: default-deny-ingress
spec:
 podSelector: {}
 policyTypes:
 - Ingress
```

Now apply the default deny network policy:

```
$ kubectl apply -f networkpolicy-default-deny.yaml \
  --namespace kuard-networkpolicy
networkpolicy.networking.k8s.io/default-deny-ingress created
```

Now let's test access to the `kuard` Pod from the test-source Pod:

```
$ kubectl run test-source --rm -ti --image busybox /bin/sh \
  --namespace kuard-networkpolicy
If you don't see a command prompt, try pressing enter.
/ # wget -q --timeout=5 kuard -O -
wget: download timed out
```

We can no longer access the `kuard` Pod from the test-source Pod due to the default deny Network Policy. Create a Network Policy that allows access from the test-source to the `kuard` Pod. Create a file called *networkpolicy-kuard-allow-test-source.yaml* with the contents of Example 19-12.

Example 19-12. networkpolicy-kuard-allow-test-source.yaml

```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: access-kuard
spec:
  podSelector:
    matchLabels:
      app: kuard
  ingress:
    - from:
      - podSelector:
          matchLabels:
            run: test-source
```

Apply the Network Policy:

```
$ kubectl apply \
  -f code/chapter-security/networkpolicy-kuard-allow-test-source.yaml \
  --namespace kuard-networkpolicy
networkpolicy.networking.k8s.io/access-kuard created
```

Again, verify that the test-source Pod can indeed access the `kuard` Pod:

```
$ kubectl run test-source --rm -ti --image busybox /bin/sh \
  --namespace kuard-networkpolicy
If you don't see a command prompt, try pressing enter.
/ # wget -q kuard -O -
<!doctype html>

<html lang="en">
<head>
  <meta charset="utf-8">

  <title><KUAR Demo></title>
...
```

Clean up the namespace by running the following command:

```
$ kubectl delete namespace kuard-networkpolicy
namespace "kuard-networkpolicy" deleted
```

Applying Network Policy provides an extra layer of security for your workloads and continues to build on the defense in depth and principle of least privilege concepts.

Service Mesh

Service mesh can also be used to increase your workload's security posture. Service meshes offer access policies, which allow the configuration of protocol-aware policies based on services. For example, your access policy might declare that ServiceA connects to ServiceB via HTTPS on port 443. In addition, service meshes typically implement mutual TLS on all service-to-service communication, which means that not only is the communication encrypted but the service identities are also verified. If you would like to learn more about service meshes and how they can be used to secure your workloads, check out Chapter 15.

Security Benchmark Tools

There are several open source tools that allow you to run a suite of security benchmarks against your Kubernetes cluster to determine if your configuration meets a predefined

set of security baselines. Once such tool is called `kube-bench`. `kube-bench` can be used to run the CIS Benchmarks for Kubernetes. Tools like `kube-bench` running the CIS Benchmarks aren't specifically focused on Pod security; however, they can certainly expose any cluster misconfigurations and help identify remediations. `kube-bench` can be run using the following command:

```
$ kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench...
job.batch/kube-bench created
```

You can then review the benchmark output and remediations via the Pod logs:

```
$ kubectl logs job/kube-bench
[INFO] 4 Worker Node Security Configuration
[INFO] 4.1 Worker Node Configuration Files
[PASS] 4.1.1 Ensure that the kubelet service file permissions are set to 644...
[PASS] 4.1.2 Ensure that the kubelet service file ownership is set to root  ...
[PASS] 4.1.3 If proxy kubeconfig file exists ensure permissions are set to  ...
[PASS] 4.1.4 Ensure that the proxy kubeconfig file ownership is set to root ...
[PASS] 4.1.5 Ensure that the --kubeconfig kubelet.conf file permissions are ...
[PASS] 4.1.6 Ensure that the --kubeconfig kubelet.conf file ownership is set...
[PASS] 4.1.7 Ensure that the certificate authorities file permissions are   ...
[PASS] 4.1.8 Ensure that the client certificate authorities file ownership  ...
[PASS] 4.1.9 Ensure that the kubelet --config configuration file has permiss...
[PASS] 4.1.10 Ensure that the kubelet --config configuration file ownership ...
[INFO] 4.2 Kubelet
[PASS] 4.2.1 Ensure that the anonymous-auth argument is set to false (Automated)
[PASS] 4.2.2 Ensure that the --authorization-mode argument is not set to    ...
[PASS] 4.2.3 Ensure that the --client-ca-file argument is set as appropriate...
[PASS] 4.2.4 Ensure that the --read-only-port argument is set to 0 (Manual)
[PASS] 4.2.5 Ensure that the --streaming-connection-idle-timeout argument is...
[FAIL] 4.2.6 Ensure that the --protect-kernel-defaults argument is set to   ...
[PASS] 4.2.7 Ensure that the --make-iptables-util-chains argument is set to ...
[PASS] 4.2.8 Ensure that the --hostname-override argument is not set (Manual)
[WARN] 4.2.9 Ensure that the --event-qps argument is set to 0 or a level    ...
[WARN] 4.2.10 Ensure that the --tls-cert-file and --tls-private-key-file arg...
[PASS] 4.2.11 Ensure that the --rotate-certificates argument is not set to  ...
[PASS] 4.2.12 Verify that the RotateKubeletServerCertificate argument is set...
[WARN] 4.2.13 Ensure that the Kubelet only makes use of Strong Cryptographic...

== Remediations node ==
4.2.6 If using a Kubelet config file, edit the file to set protectKernel...
If using command line arguments, edit the kubelet service file
/etc/systemd/system/kubelet.service.d/10-kubeadm.conf on each worker node and
```

```
set the below parameter in KUBELET_SYSTEM_PODS_ARGS variable.
--protect-kernel-defaults=true
Based on your system, restart the kubelet service. For example:
systemctl daemon-reload
systemctl restart kubelet.service

4.2.9 If using a Kubelet config file, edit the file to set eventRecordQPS...
If using command line arguments, edit the kubelet service file
/etc/systemd/system/kubelet.service.d/10-kubeadm.conf on each worker node and
set the below parameter in KUBELET_SYSTEM_PODS_ARGS variable.
Based on your system, restart the kubelet service. For example:
systemctl daemon-reload
systemctl restart kubelet.service
...
```

Using tools like `kube-bench` with the CIS benchmarks can help identify whether your Kubernetes cluster meets a security baseline and provide remediations if needed.

Image Security

Another important part of Pod security is keeping the code and application within the Pod secure. Securing an application's code is a complex topic beyond the scope of this chapter; however, the basics for container image security include making sure that your container image registry is doing *static scanning* for known code vulnerabilities. Additionally, you should have a tool for doing *runtime scanning* that identifies vulnerabilities that have been discovered after an image started running and also looks for potentially malicious activity like intrusions. There are many scanning tools provided by both open source and proprietary companies. In addition to security scanning, focusing on minimizing the contents of your container image to remove unnecessary dependencies minimizes the noise from this scanning. Finally, image security is another great reason to invest in continuous delivery so that you can rapidly patch and redeploy an image when vulnerabilities are found.

Summary

In this chapter, we covered many different security-focused APIs and resources that can be used to improve the security posture of your workloads. By practicing defense in depth and principle of least privilege, you can incrementally improve the baseline security of your Kubernetes cluster. It's never too late to start practicing better security, and

this chapter provides everything you need to be confident that you have an understanding of the security controls Kubernetes offers.

Previous chapter

< 18. Accessing Kubernetes from Common Programming Languages

Next chapter

20. Policy and Governance for Kubernetes Clusters >